Introduction to Artificial Intelligence

Lecture 2 · Heuristic search

Greedy search · A* · designing heuristics

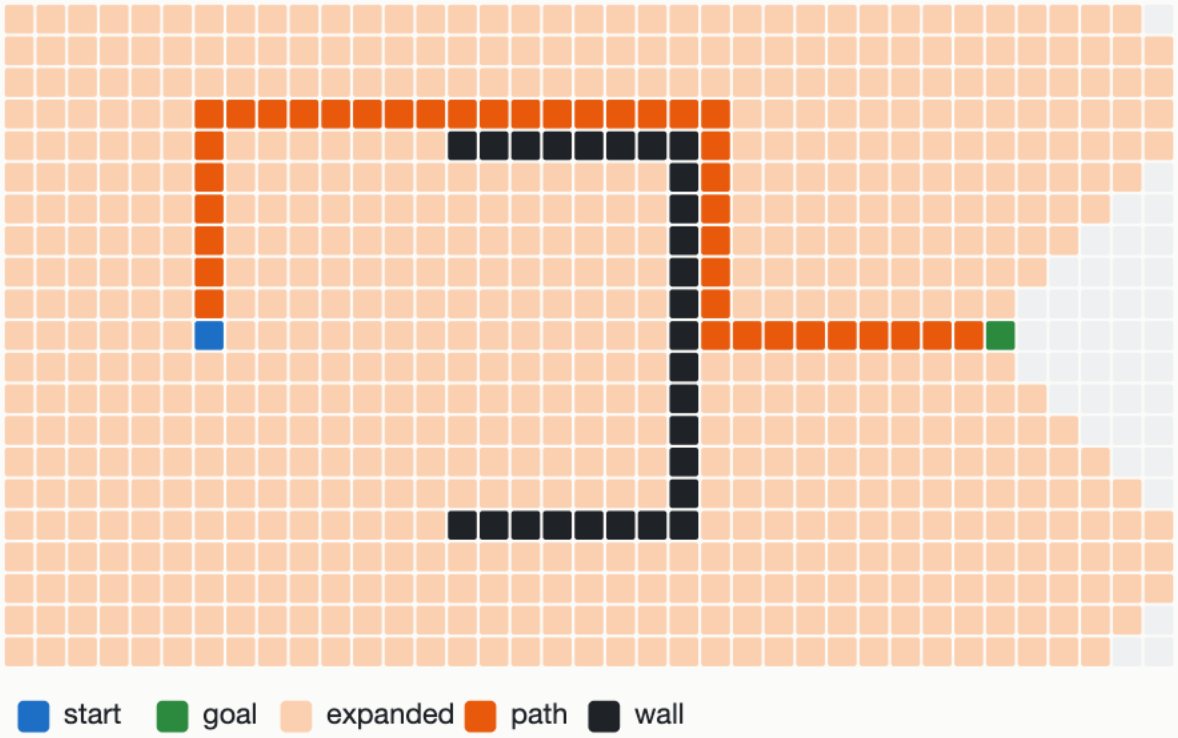
Today

01	**Informed search**	Heuristic functions, best-first search, greedy search
02	**A***	The algorithm, admissibility, consistency and optimality
03	**Designing heuristics**	The 8-puzzle, dominance, relaxed problems
04	**Heuristic search in Python**	Heuristics as functions, A* in the framework

01

Informed search

Blind search looks everywhere



Uniform-cost search expands 710 of the 750 free cells to find a path of cost 39: it has no idea where the goal is. If we knew roughly where the goal is, where should we look first?

Heuristic functions

Definition. A *heuristic* h(n) estimates the cost of the cheapest path from the state of node n to a goal. It depends only on the state, and h(n) = 0 at a goal.

Road map

the straight-line distance to the goal

Grid or maze

the Manhattan distance: $|\Delta\text{row}| + |\Delta\text{col}|$

8-puzzle

the number of misplaced tiles

A heuristic is knowledge about the domain, given to the search as a function.

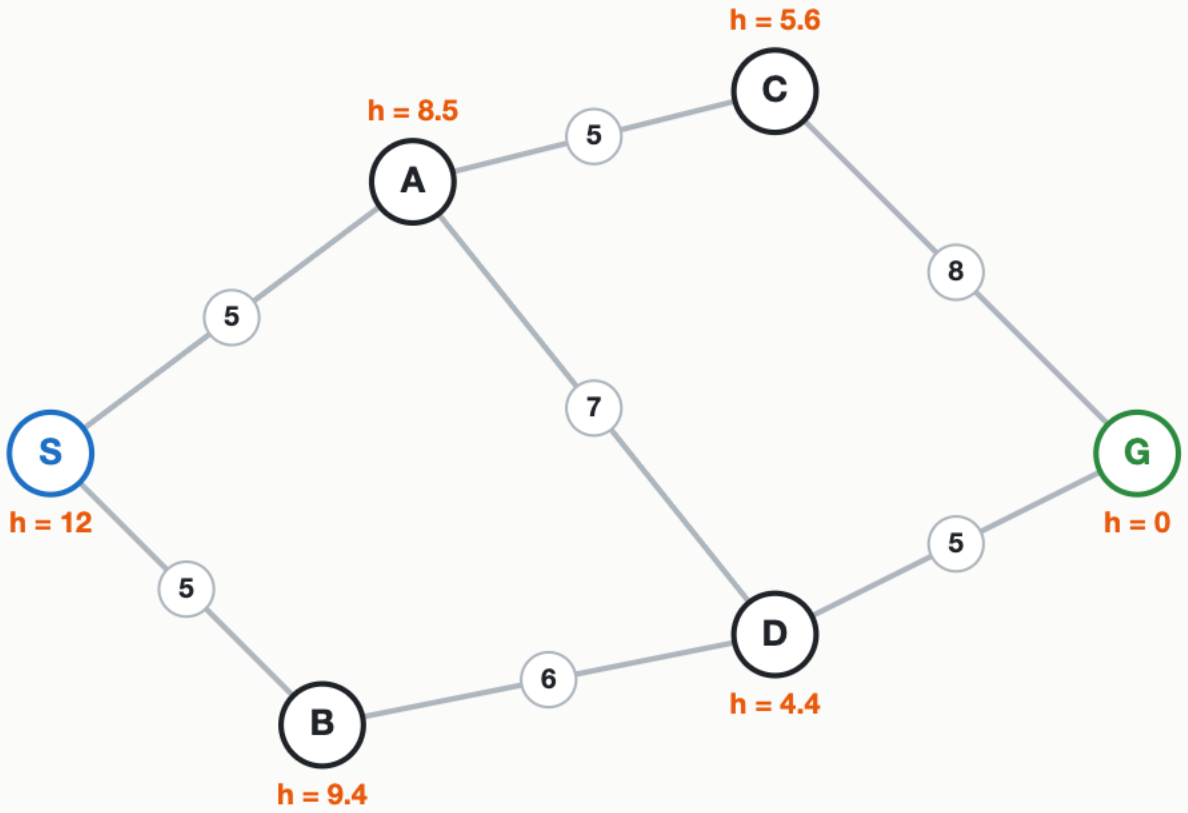
Best-first search

The generic graph search of lecture 1, with a priority queue ordered by an *evaluation function* f(n): always expand the node with the lowest f.

Strategy	f(n)	Expands first
Uniform-cost search	g(n)	the node with the cheapest path so far
Greedy best-first search	h(n)	the node that seems closest to a goal
A*	g(n) + h(n)	the node on the cheapest estimated solution

Only f changes: the algorithm is the same.

An example: a small road map



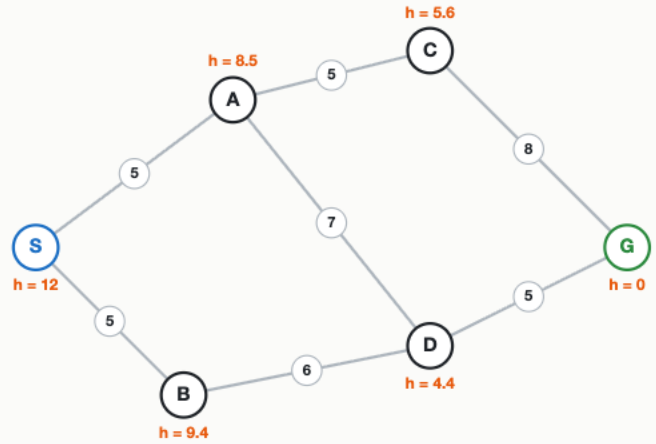
Numbers on roads: distances. h: straight-line distance to G.

- Find the cheapest route from S to G.
- The numbers on the roads are their lengths.
- h(n) is the straight-line distance from n to G: no route can be shorter.

Greedy best-first search

Expand the node that seems closest to the goal: f(n) = h(n).

Step	Removed	Added to the frontier (h)
1	S	A (8.5), B (9.4)
2	A	C (5.6), D (4.4)
3	D	G (0)
4	G	goal: S → A → D → G, cost 17



Numbers on roads: distances. h: straight-line distance to G.

The cheapest route costs 16. Greedy search went to D through A because D *looked* close, ignoring what it had already paid.

Greedy search: guarantees

- **Not optimal**: it ignores g(n), the cost already paid.
- **Complete** in finite state spaces, as graph search; in infinite ones it can follow a misleading path forever.
- Time and space $O(b^m)$ in the worst case; with a good heuristic, often far less.
- On the lab maze, greedy search returns a path of 70 steps, while the shortest has 62.

02

A*

A* search

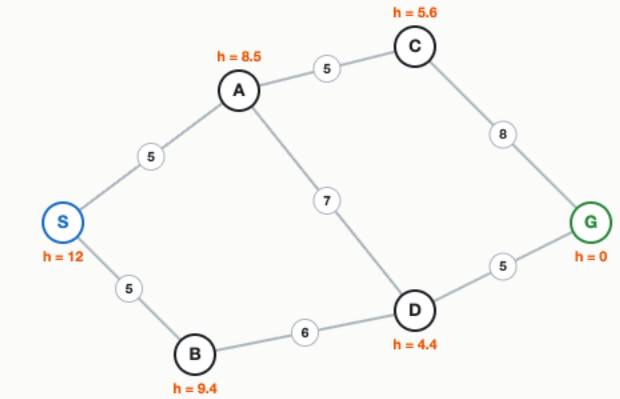
$$f(n) = g(n) + h(n)$$

- g(n): the cost of the path to n, known exactly.
- h(n): the estimated cost from n to a goal.
- f(n): the estimated cost of the cheapest solution through n.

Uniform-cost search uses only g, and greedy search only h. A* combines both.

A* on the road map

Step	Removed (f)	Added or improved: g + h = f
1	S (12)	A: 5 + 8.5 = 13.5; B: 5 + 9.4 = 14.4
2	A (13.5)	C: 10 + 5.6 = 15.6; D: 12 + 4.4 = 16.4
3	B (14.4)	D, improved: 11 + 4.4 = 15.4
4	D (15.4)	G: 16 + 0 = 16
5	C (15.6)	G through C would cost 18: no improvement
6	G (16)	goal: S → B → D → G, cost 16, the optimum



Numbers on roads: distances. h: straight-line distance to G.

Like uniform-cost search, A* tests for the goal when a node is *removed*: G was in the frontier from step 4, but only leaves it once nothing cheaper is left.

Admissible heuristics

Definition. A heuristic is *admissible* if it never overestimates the cost to reach a goal:

$$h(n) \leq h^*(n)$$

where h*(n) is the true cost of the cheapest path from n to a goal.

- Straight-line distance: no road is shorter than a straight line.
- Manhattan distance, on a grid with unit moves up, down, left and right.
- Misplaced tiles: every misplaced tile needs at least one move.
- *Not* admissible: twice the straight-line distance.

A* is optimal

> **Theorem.** With an admissible heuristic, A* (tree-like search) returns an optimal solution.

Proof sketch. Let C* be the optimal cost, and suppose a goal node G_2 with $g(G_2) > C^*$ is in the frontier. Some node n on an optimal path is in the frontier too, since the frontier separates. Then

$$f(n) = g(n) + h(n) \;\leq\; g(n) + h^*(n) = C^* \;<\; g(G_2) = f(G_2)$$

so n leaves the frontier before G_2, and so does every later node of that optimal path, up to its goal. ∎

Consistent heuristics

Definition. A heuristic is *consistent* if h(G) = 0 at goals and, for every node n and every successor n′ reached by action a,

$$h(n) \leq c(n, a, n') + h(n')$$

A triangle inequality: the estimate cannot drop by more than the cost of a step.

Consistent implies admissible. By induction on the number of steps to a goal along a cheapest path: if n′ is its first step, then h(n) ≤ c(n, a, n′) + h(n′) ≤ c(n, a, n′) + h*(n′) = h*(n). ∎

Straight-line distance, Manhattan distance and misplaced tiles are all consistent.

A* with graph search

> **Theorem.** With a consistent heuristic, A* with graph search is optimal, and removes each state from the frontier at most once.

Proof sketch.

1. f never decreases along a path:

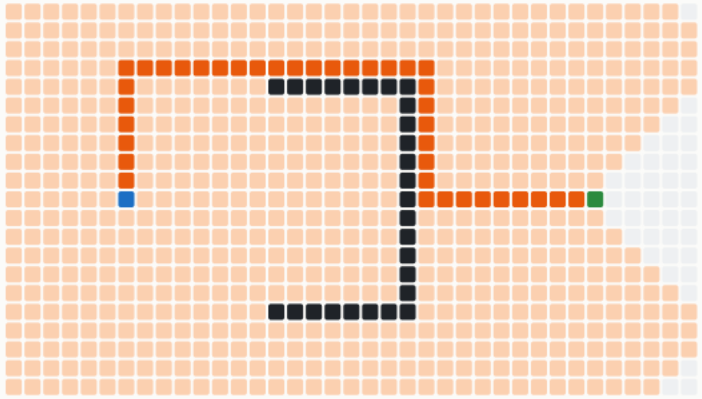
$$f(n') = g(n) + c(n, a, n') + h(n') \;\geq\; g(n) + h(n) = f(n)$$

2. So nodes leave the frontier in nondecreasing order of f, as with g in uniform-cost search.
3. The first time a state leaves the frontier, it is through a cheapest path; so the first goal removed is optimal. ∎

Our graph search also reopens a state when it finds a cheaper path, so it stays optimal with any admissible heuristic; consistency guarantees that this never happens.

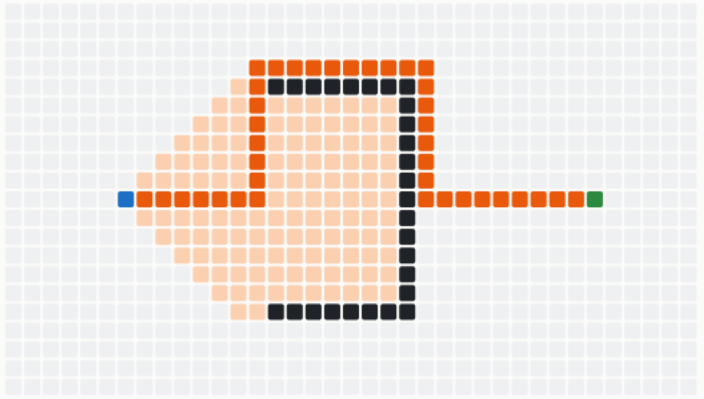
What A* expands

Uniform-cost search



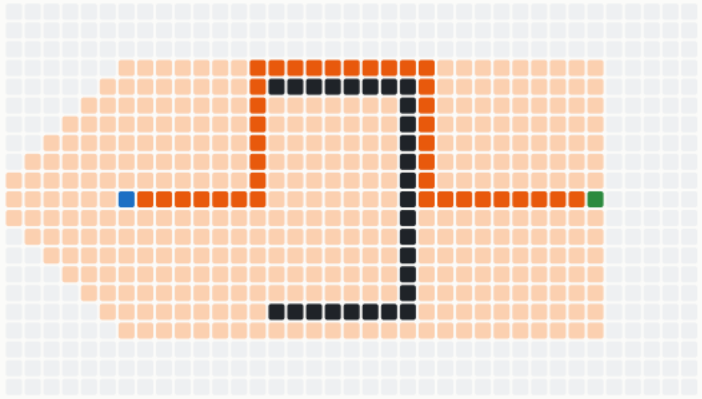
710 expanded, path cost 39

Greedy search



164 expanded, path cost 39

A*



410 expanded, path cost 39

- A* expands every node with f(n) < C*, possibly some with f(n) = C*, and none with f(n) > C*.
- Greedy search expands the fewest here, but in general without any guarantee on the solution.

The price of A*

- **Memory**: like uniform-cost search, A* keeps every node it generates, and memory runs out first.
- **Optimally efficient**: with a consistent heuristic, no other optimal algorithm that extends paths from the start expands fewer nodes, apart from ties.
- Its cost grows quickly with the errors of h: a better heuristic matters more than a faster computer.
- **Variants**: IDA*, iterative deepening on f with linear memory, and weighted A*.

Weighted A*

$$f(n) = g(n) + w \cdot h(n), \qquad w > 1$$

- Trusts the heuristic more: it expands fewer nodes, but may return a worse solution.
- With an admissible heuristic, the solution costs at most w · C*.
- On the grid of the previous slides, w = 2 expands 212 nodes instead of 410, and still finds a path of cost 39.

03

Designing heuristics

Two heuristics for the 8-puzzle



The true distance from this board to the goal is 20 moves.

- h_1, misplaced tiles: each needs at least one move.
- h_2, the sum of the Manhattan distances of the tiles to their goal squares: each tile needs at least that many moves.
- Both are admissible: here $h_1 = 6$ and $h_2 = 14$, while the true distance is 20.

How much do they help?

Depth	Breadth-first search	A* with h_1	A* with h_2
4	24	4	4
8	222	18	12
12	1,485	96	32
16	9,787	568	116
20	48,640	3,522	533
24	129,875	18,654	1,712

Average number of nodes expanded on 40 random boards at each depth (all 16 boards at depth 4). Every A* solution was optimal.

Effective branching factor

$$N + 1 = 1 + b^* + (b^*)^2 + \cdots + (b^*)^d$$

If a search expands N nodes to find a solution at depth d, b* is the branching factor that a uniform tree of depth d needs to have N + 1 nodes. The closer to 1, the more focused the search.

Depth	Breadth-first search	A* with h_1	A* with h_2
8	1.77	1.18	1.08
12	1.71	1.30	1.14
16	1.68	1.37	1.21
20	1.64	1.41	1.27
24	1.57	1.43	1.28

Breadth-first search drops at depth 24 only because it starts to exhaust the 181,440 reachable boards.

Dominance

> If $h_2(n) \geq h_1(n)$ for every n, and both are admissible, h_2 *dominates* h_1: A* with h_2 never expands more nodes than A* with h_1, apart from ties.

Why. With a consistent heuristic, A* expands every node with $f(n) < C*$, that is, with $h(n) < C* - g(n)$. A larger heuristic satisfies this for fewer nodes. ∎

For the 8-puzzle, $h_2 \geq h_1$, since every misplaced tile is at distance at least 1. Prefer the larger heuristic, as long as it stays admissible and is fast to compute.

Heuristics from relaxed problems

A *relaxed* problem has fewer restrictions on its actions. The 8-puzzle rule: a tile can move from square A to square B if A is adjacent to B **and** B is blank.

Relaxed rule: a tile can move from A to B...	**Optimal cost of the relaxed problem**
if A is adjacent to B	h_2, the Manhattan distance
always	h_1, the number of misplaced tiles

Theorem. The optimal cost of a relaxed problem is an admissible and consistent heuristic for the original: every solution of the original is a solution of the relaxed problem, which only has more actions.

Combining heuristics

$$h(n) = \max\{ h_1(n), \ldots, h_k(n) \}$$

- If every h_i is admissible, or consistent, so is their maximum.
- The maximum dominates each h_i: at every node it uses the best estimate available.
- **Pattern databases**: store the exact cost of solving part of the puzzle (say, tiles 1 to 4) for every arrangement of that part; looking it up gives an admissible heuristic.

04

Heuristic search in Python

A* in the framework

```
Heuristic = Callable[[S], float]


def best_first_search(problem: SearchProblem[S],
                      f: Callable[[Node[S]], float]) → Optional[Node[S]]:
    """Graph search that always expands the node with the lowest f."""
    return graph_search(problem, PriorityFrontier(f))


def greedy_search(problem: SearchProblem[S], h: Heuristic[S]) → Optional[Node[S]]:
    return best_first_search(problem, lambda n: h(n.state))


def a_star_search(problem: SearchProblem[S], h: Heuristic[S]) → Optional[Node[S]]:
    return best_first_search(problem, lambda n: n.path_cost + h(n.state))
```

Only the priority function changes: the strategy pattern of lecture 1, now with functions as arguments. Uniform-cost search is **best_first_search(problem, lambda n: n.path_cost)**.

Heuristics are functions

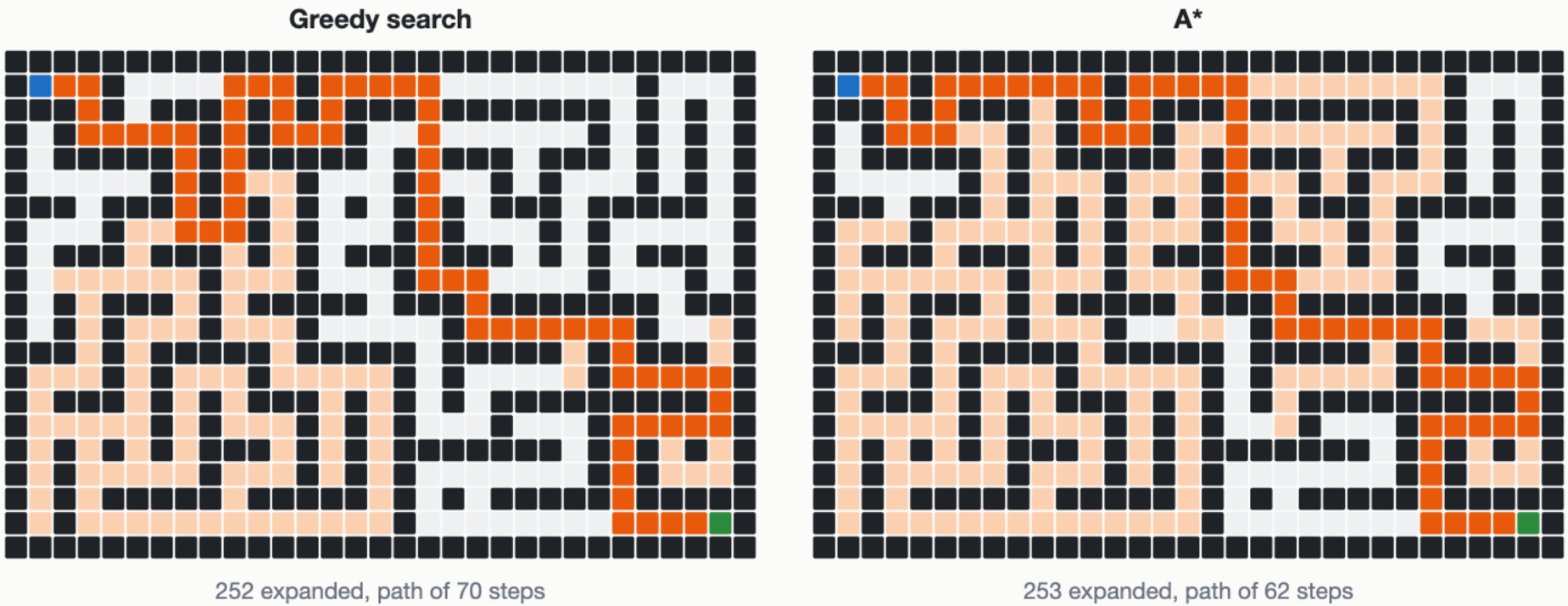
```
def manhattan_to(goal: tuple[int, int]) → Heuristic[MazeState]:
    """The Manhattan distance to goal, as a heuristic for the maze."""
    def h(state: MazeState) → float:
        return abs(state.row - goal[0]) + abs(state.col - goal[1])
    return h


def max_h(*hs: Heuristic[S]) → Heuristic[S]:
    """The pointwise maximum of several heuristics: a new heuristic."""
    return lambda state: max(h(state) for h in hs)


maze = MazeProblem()
node = a_star_search(maze, manhattan_to(maze.exit))
```

- **manhattan_to(goal)** returns a new function that remembers **goal**: a *closure*.
- **max_h** takes heuristics and returns a heuristic: a *higher-order* function.

A* on the lab maze



252 expanded, path of 70 steps

253 expanded, path of 62 steps

Breadth-first search: 279 nodes expanded and 62 steps. In a maze, walls make the Manhattan distance a weak estimate: A* barely beats breadth-first search, and greedy search is led astray.

Summary

- A heuristic h(n) estimates the remaining cost; best-first search expands the node with the lowest f(n).
- Greedy search, f = h, is fast but not optimal.
- A*, f = g + h, is optimal with an admissible heuristic (tree search) or a consistent one (graph search), and expands every node with f < C*.
- Good heuristics come from relaxed problems; a dominating heuristic expands fewer nodes, and the maximum of admissible heuristics is admissible.
- In Python, heuristics and strategies are functions passed to one generic search.

In the lab

1. Implement h_1 and h_2 for the 8-puzzle, and run A* with each on boards of increasing depth. How many nodes does each expand?
2. Prove that h_2 is admissible and consistent.
3. Try weighted A* with w = 1, 2 and 5. How do the number of nodes and the solution length change?
4. Run A* on the lab maze with the Manhattan distance, and compare it with breadth-first search.

Next: logic programming

- A Prolog program is a set of facts and rules; running a query is a **search for a proof**.
- Prolog explores that search space with **depth-first search and backtracking**.
- We will see what this implies: very short programs for search problems, and programs that never terminate.

Reading: Russell and Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, sections 3.5 and 3.6.